Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: Varun S | Roll No: 241401124

Aim

To explore how pro-code capabilities can extend a low-code Zoho Creator application using Zoho Catalyst / custom functions, by creating and executing a custom Deluge function in the Insurance Claim Management application, which handles insurance claim requests and their processing.

Introduction

Low-code platforms allow applications to be developed using visual components such as forms, reports and workflows, with little hand-written code. This makes it quick to build standard business applications, such as an application that collects insurance claim requests from policyholders and tracks them until they are settled.

When the standard low-code functionality is insufficient for a particular requirement, pro-code extensions allow developers to introduce programmable logic into the application. Zoho Creator supports this through Deluge, its scripting language, and through custom functions, which are reusable units of Deluge code that can be called within an application when required. Zoho Catalyst, Zoho's serverless development platform, represents a further pro-code option for requirements that go beyond what can be scripted inside the application.

In this experiment, a simple custom function named processClaimRequest was created for the Insurance Claim Management application and executed to observe how a pro-code extension is created and run in Zoho Creator. The function accepts two arguments, claimantName and claimId, and returns a string.

Pro-Code Extension in Zoho Creator

Extending a low-code application with custom functions means that the visual structure of the application (forms, reports and workflows) is retained, while specific logic is written as code in a function and used where required. The main concepts are as follows:

- **Custom functions:** named, reusable functions created in the Functions section of the Workflow area. A function has a name, a namespace, a return type and optional arguments.
- **Deluge scripting:** the scripting language of Zoho Creator, in which the logic of a custom function is written. The Deluge editor provides a list of ready-made statements such as conditions, data-access operations, set variable, call function, send mail and return.
- **Reusable application logic:** a function is written once and can be called from within the application whenever its logic is needed, which avoids repeating the same code.
- **Custom business logic:** application-specific rules, for example claim eligibility checks, that are not covered by standard configuration can be implemented in a function.
- **Data processing:** Deluge provides operations to add, fetch, aggregate, update and delete records, which allow claim records to be processed through code.
- **Automation and extensibility:** functions allow automation to be extended beyond standard options, so that the application can be adapted to specific requirements.

241401124

Figure 1 illustrates the concept. A user works with the Insurance Claim Management application, which has components such as forms, workflows, reports and custom functions. The pro-code extension (Deluge scripting, custom functions, business logic, data processing and automation) is represented by the custom function processClaimRequest(), which accepts the claimant name and claim ID, runs Deluge logic and produces a return value.

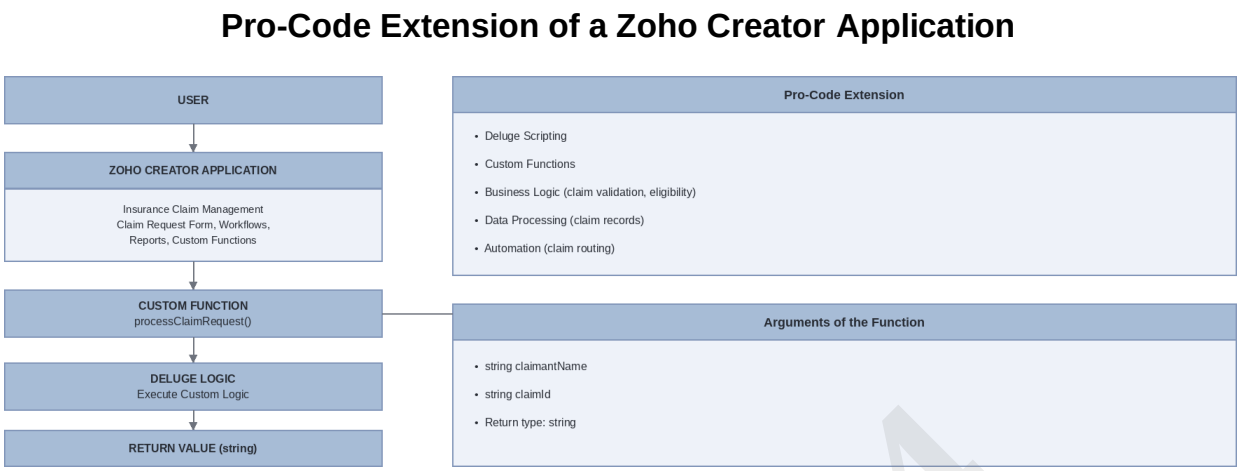


Figure 1: Pro-Code Extension of a Zoho Creator Application (Supporting Diagram)

The diagram is a conceptual illustration. In this experiment, only the creation and execution of the custom function were carried out; the function was not attached to a workflow or connected to any other component.

Procedure

1 Open the Insurance Claim Management application in Zoho Creator in Edit mode. The Design view lists the forms (Claim_Request, Policy_Holder, Claim_Assessment and Settlement) and the reports of the application (Figure 2).

2 Navigate to the Workflow section.

3 Open the Functions section. As no function exists yet, the page shows the message “Execute re-usable functions that can be called within an application when required” with the Create Function option (Figure 3).

24140124

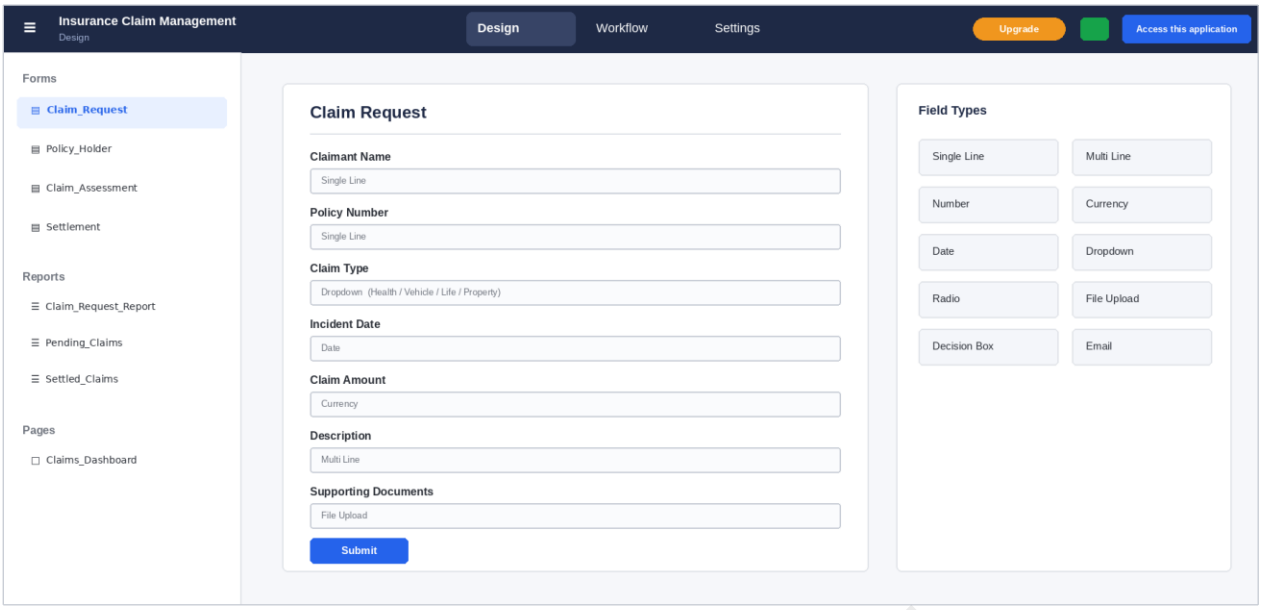


Figure 2: Design View of the Insurance Claim Management Application (Claim Request Form)

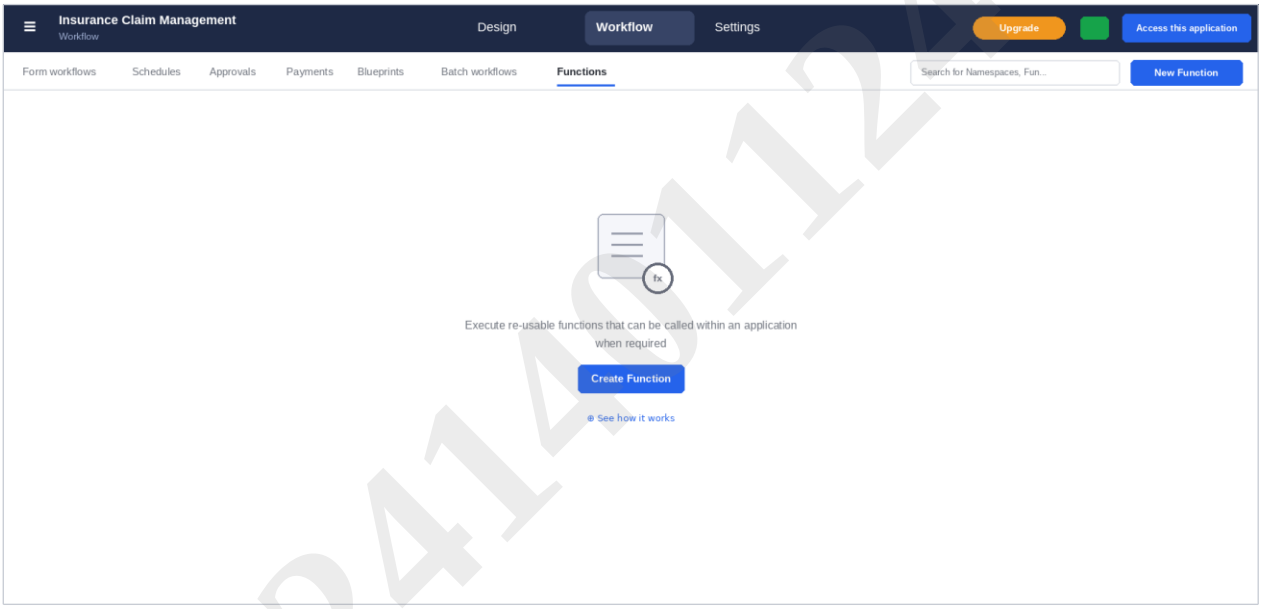


Figure 3: Functions Section of the Workflow Area

4 Select Create Function. The “Execute a function when called” form is displayed, with the fields Name the function, Select the language, Select namespace, Select return type and Specify arguments.

5 Enter the function name processClaimRequest.

6 Select Deluge as the programming language.

7 Use the Default namespace.

8 Set the return type as string.

9 Under Specify arguments, add two arguments of type string: claimantName and claimId. Then select Create Function (Figure 4).

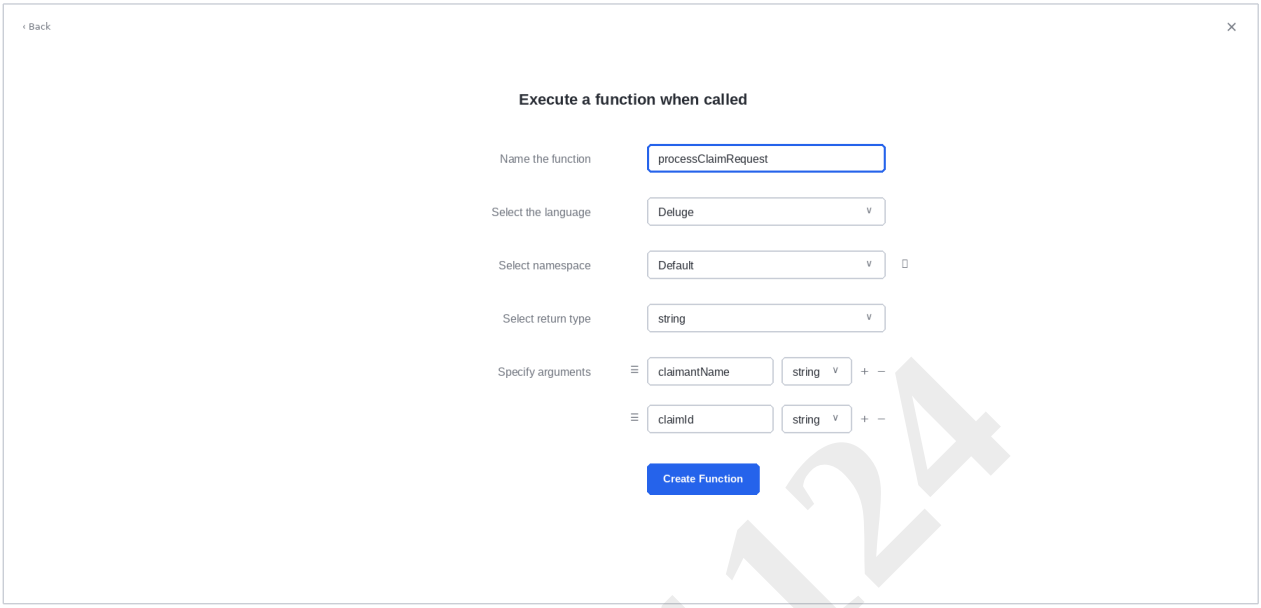


Figure 4: Create Function Form – Name, Language, Namespace, Return Type and Arguments

10 The Deluge editor opens with the function declaration string processClaimRequest(string claimantName, string claimId) generated from the selected options. Add the Deluge logic to return the claim registration message (shown in the next section), in place of the empty return value generated by default (Figure 5a).

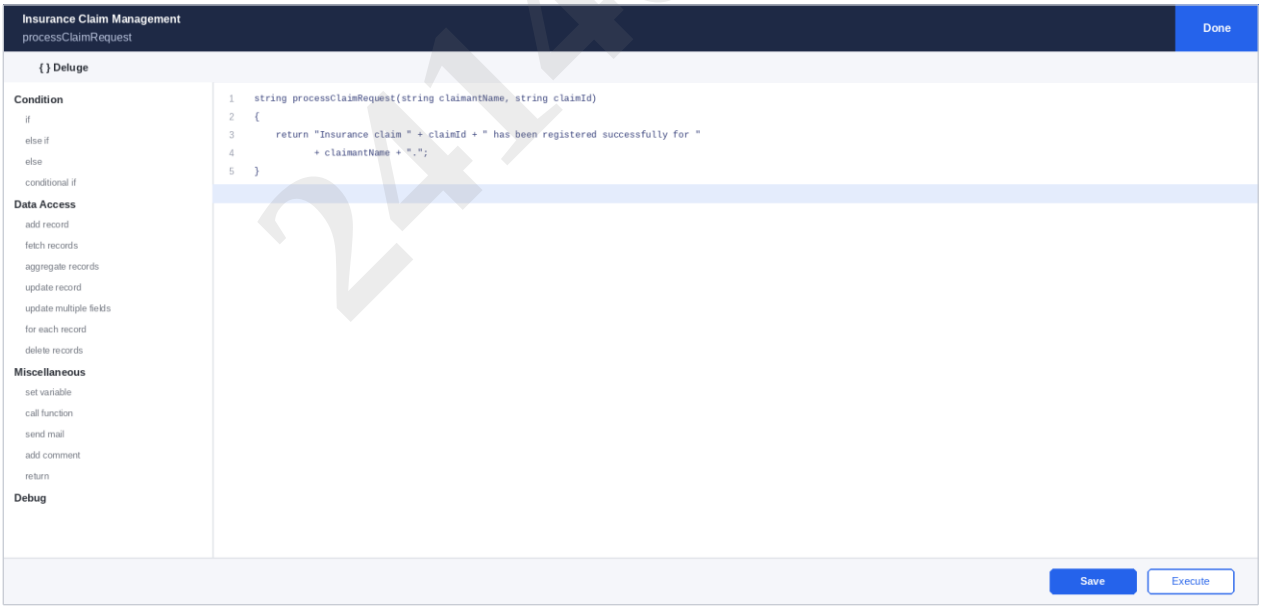


Figure 5(a): Deluge Editor – Function Declaration and Logic for processClaimRequest()

24140124

11 Save the custom function using Save.

12 Execute the function using Execute. The Execute window asks for the values of the input arguments; enter Arun for claimantName and CLM101 for claimId, and select Submit (Figure 5b).

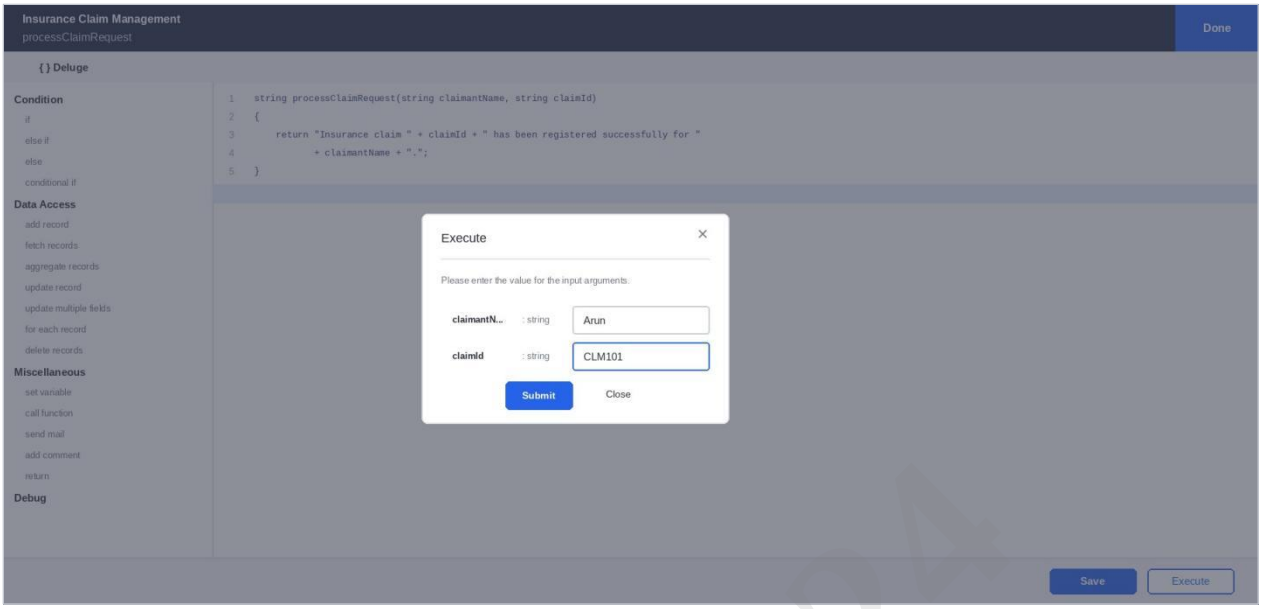


Figure 5(b): Execute Window – Entering the Input Arguments

13 Verify that the Execute window reports Executed Successfully and that the return value is the expected message (Figure 6). The saved function is then listed in the Functions section (Figure 7).

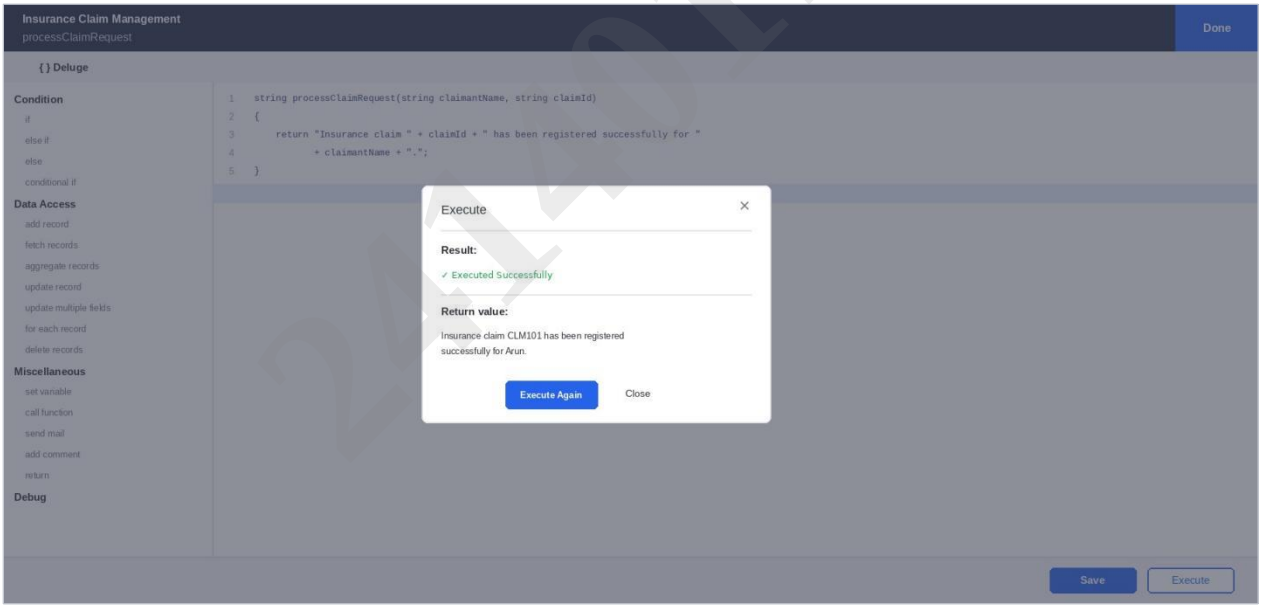


Figure 6: Execution of processClaimRequest() Custom Function

Custom Function Implementation

The following Deluge code was used for the custom function:

```
string processClaimRequest(string claimantName, string claimId)
{
    return "Insurance claim " + claimId + " has been registered successfully for "
           + claimantName + ".";
}
```

The function is declared with the return type string and takes two string arguments, claimantName and claimId. When it is called, it returns the string “Insurance claim ” followed by the claim ID supplied, the text “ has been registered successfully for ” and the claimant name supplied, confirming that the claim request was registered. Both arguments are therefore used by the logic of the function. The function details are summarised below.

Property	Value
Function Name	processClaimRequest
Language	Deluge
Namespace	Default
Return Type	string
Arguments	string claimantName, string claimId
Test Input	claimantName = Arun, claimId = CLM101
Return Value	Insurance claim CLM101 has been registered successfully for Arun.

Function Execution

The custom function was executed in Zoho Creator using the Execute option of the Deluge editor, with the values Arun and CLM101 supplied as arguments. The Execute window displayed the result Executed Successfully and the return value “Insurance claim CLM101 has been registered successfully for Arun.”, which is the expected confirmation message. The code shown behind the window is the saved function, with the message in the return statement.

Saved Custom Function

After saving, the function appears in the Functions section of the Workflow area of the Insurance Claim Management application. It is listed under the Default namespace with the function name processClaimRequest, the language Deluge, the return type string and the syntax processClaimRequest(string claimantName, string claimId).

24401124

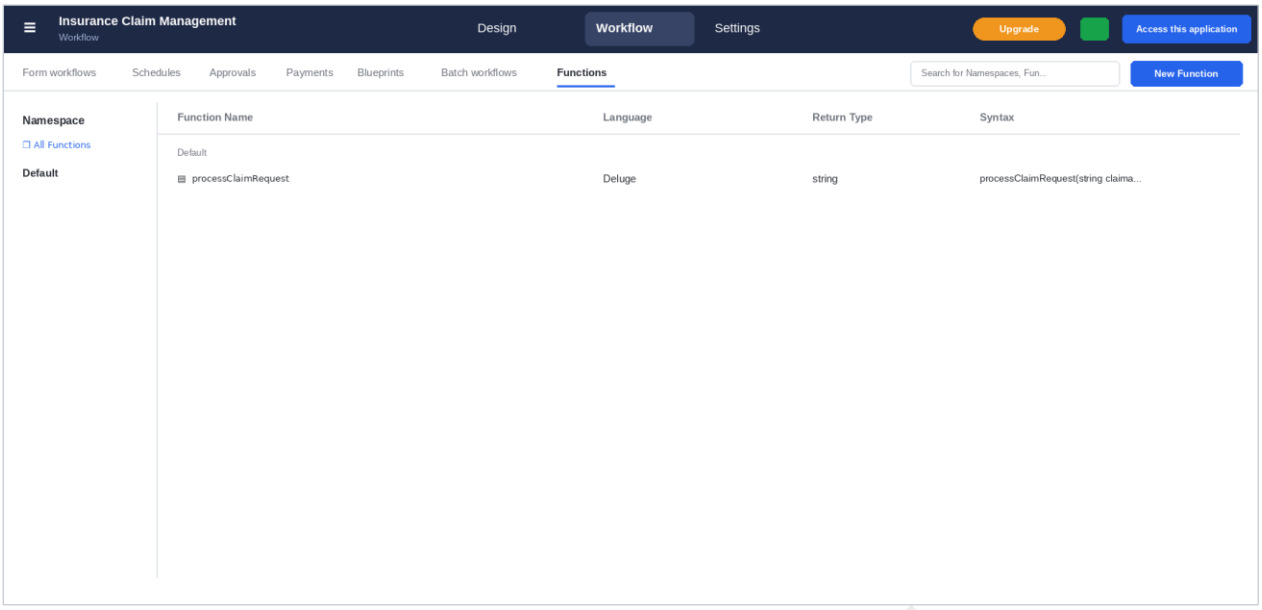


Figure 7: Saved processClaimRequest Custom Function

Execution Flow

Figure 8 shows the execution flow of the custom function. The flow is: Insurance Claim Management Application → processClaimRequest() → Deluge Function Execution → Custom Logic → String Return Value → Successful Execution.

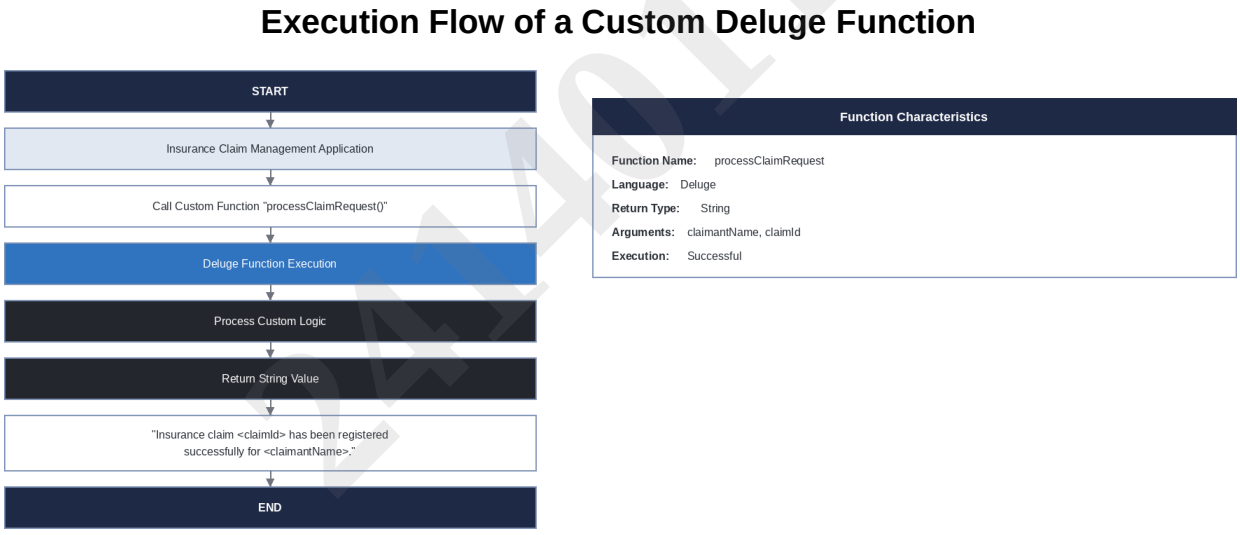


Figure 8: Execution Flow of a Custom Deluge Function (Supporting Diagram)

- The application calls the custom function processClaimRequest() with the arguments claimantName and claimId.
- The Deluge function is executed and its custom logic is processed. In this experiment the logic consists of returning a message that includes the claim ID and the claimant name.
- A string value is returned, and the function completes with successful execution, as confirmed in Figure 6.

Low-Code and Pro-Code Extension

Low-code components provide the basic structure of an application, while pro-code extensions allow additional programmable logic and customisation. Figure 9 presents the two together as a combined approach.

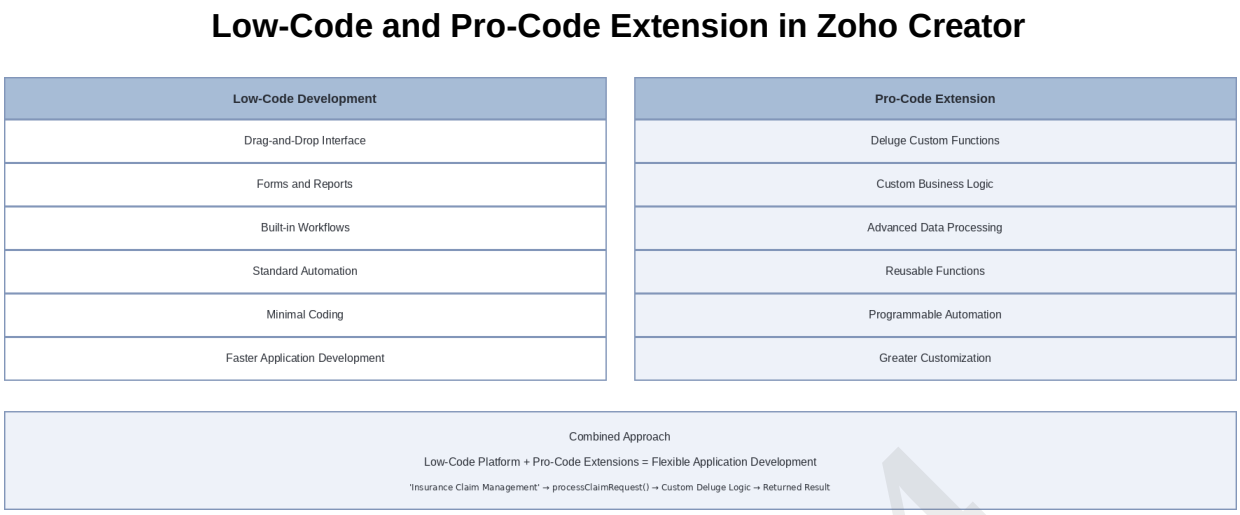


Figure 9: Low-Code and Pro-Code Extension in Zoho Creator (Supporting Diagram)

Low-Code	**Pro-Code**
Forms	Deluge functions
Reports	Custom logic
Workflows	Advanced processing
Standard automation	Reusable functions
Visual development	Programmable automation

The comparison describes the general roles of the two approaches. This experiment demonstrated only a simple custom function that returns a confirmation message; it did not implement advanced processing or integration with workflows or other systems.

2414012 4

Pro-Code Extension in Insurance Claim Processing

An insurance claim passes through several stages, from the moment a policyholder submits a claim request until the amount is settled. The low-code part of the Insurance Claim Management application (the Claim_Request form, the reports and the dashboard) captures and displays this information. Figure 10 shows the stages of a typical claim and the stage at which a function such as processClaimRequest() can be applied.

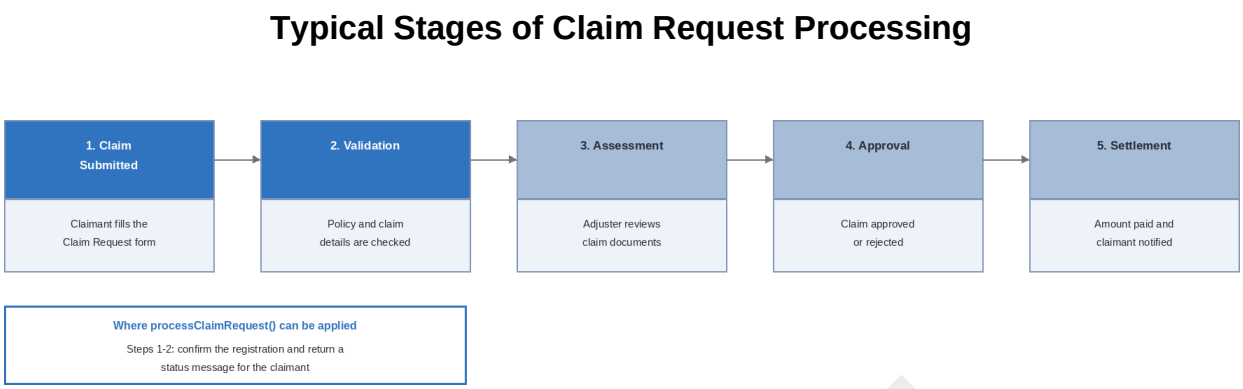


Figure 10: Stages of Insurance Claim Processing (Conceptual Diagram)

Custom functions of this kind can be extended to support requirements that standard forms and workflows do not cover easily. Possible examples are listed below; none of them was implemented in this experiment.

- **Claim validation:** checking that the policy number is valid and that the claim amount does not exceed the sum insured.
- **Data processing:** fetching the earlier claims of a policyholder with the fetch records statement and counting them with aggregate records.
- **Notification:** using the send mail statement to inform the claimant that the claim request has been received.
- **Reusability:** calling the same function from different forms or workflows, so that the logic is maintained at a single place.

Observations

- The function was created without changing any form or report of the application, which shows that pro-code logic is added alongside the low-code structure.
- The Execute option allows a function to be tested with sample values before it is used anywhere in the application.
- The function in this experiment performs no data processing; the registration message is only returned as a string.

24140124

Result

The custom Deluge function processClaimRequest was successfully created in the Insurance Claim Management application using the Default namespace, the string return type and the arguments claimantName and claimId. The function was saved, listed in the Functions section and executed with the test values Arun and CLM101, returning the expected confirmation message.

Conclusion

Low-code platforms such as Zoho Creator simplify application development by providing forms, reports and workflows that can be assembled visually. Where specific requirements go beyond the standard components, pro-code extensions provide additional flexibility, and custom Deluge functions can introduce application-specific logic in a reusable form.

In this experiment, a custom function named processClaimRequest was created with the Deluge language, the Default namespace, the string return type and two string arguments in the Insurance Claim Management application. The function was saved and executed successfully, returning the expected confirmation message for the supplied claimant name and claim ID. Although the function was simple, it illustrates the procedure for creating, executing and verifying a custom function. Combining low-code development with suitable pro-code extensions can support more flexible application development, in which standard functionality is built visually and specific logic, such as claim validation and processing, is added through code where it is required.

24140124